

UCS503P - Software Engineering Laboratory

Intrinsic: A DCF Valuation and Scenario-Analysis Platform

Interactive, Auditable, and Repeatable Company Valuation using Discounted Cash
Flow

Submitted by:

1024030790 Aatish Kumar Sahu

1024030811 Bhavin Bhatti

1024030814 Bhoomi Mittal

Group: 3C55 BE Third Year, CSE

Submitted to:

Ms. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.2	Project Objectives	6
2	Methodology and Design	7
2.1	System Architecture	7
2.1.1	Use Case Diagram	7
2.1.2	Class Diagram	7
2.1.3	Sequence Diagram	10
2.1.4	Data Flow Diagrams	11
2.1.5	Swimlane Diagram	11
2.1.6	Activity Diagram	11
2.1.7	High-Level Design	11
2.1.8	Technical Stack	15
2.2	Implementation Details	15
2.2.1	Module 1: Core Functionality	15

Introduction

1.1 Background and Context

Company valuation is often performed using spreadsheet-based financial models. Although spreadsheets are flexible, they make it difficult to audit the sequence of calculations, reuse previous analyses, validate financial inputs, or compare alternative assumptions systematically. The Intrinsic project addresses this problem by converting a standard Discounted Cash Flow (DCF) valuation into a structured web application in which assumptions, intermediate calculations, saved scenarios, and final valuation outputs are explicit and reproducible.

The project is intentionally positioned as a software-engineering problem rather than a finance-research problem. The DCF equations are standard; the main engineering contribution is the workflow around them: typed data contracts, deterministic valuation logic, validation, persistence, authentication infrastructure, scenario management, comparison, and automated testing. The project proposal describes the intended platform as a valuation-to-comparison workflow in which a user can enter assumptions manually, obtain a DCF valuation, save named Bull/Base/Bear scenarios, and eventually supplement manual inputs with validated external financial data.

1.1.1 Problem Statement

Traditional spreadsheet valuation workflows present four practical limitations. First, they are opaque: the final value can be inspected, but tracing every assumption and intermediate computation is difficult. Second, analyses are often stored as isolated files, so there is little structured reuse or comparison. Third, financial data is frequently copied manually from public sources, creating opportunities for transcription or data-quality errors. Finally, optimistic, base, and pessimistic scenarios are commonly represented by duplicated spreadsheets rather than a systematic comparison model.

Intrinsic addresses these limitations by making the valuation state explicit. A scenario contains a named set of assumptions together with its data provenance and, when calculated, the complete valuation output. The current implementation also provides a persistent scenario store and a side-by-side comparison view, establishing the foundation for a reusable valuation workflow.

1.2 Project Objectives

1. **Implement a deterministic DCF valuation engine:** calculate projected free cash flows, discount factors, present values, terminal value, enterprise value, equity value, and intrinsic value per share from a well-defined assumption set.
2. **Provide an auditable web workflow:** expose assumptions and intermediate valuation results in the frontend so that a user can inspect how the final value is obtained rather than receiving only a black-box number.
3. **Support scenario persistence and comparison:** allow named valuation cases to be saved, loaded, deleted, and compared across key assumptions and valuation outputs.
4. **Establish a maintainable full-stack foundation:** keep the frontend and backend synchronized through explicit contracts, persist application data through a database layer, and protect future account-specific functionality through an authentication layer.

Methodology and Design

2.1 System Architecture

The current implementation follows a layered web architecture. The React/TypeScript frontend is responsible for collecting assumptions and presenting valuation and scenario results. The FastAPI backend provides the HTTP boundary and persistence operations. SQLite/SQLModel form the current persistence layer. Authentication is implemented as a separate backend slice using password hashing and JWT access tokens. The DCF engine remains deliberately isolated from both the UI and database so that the financial calculation remains deterministic and directly testable.

The most recently updated branch used for this report is `demo/comparison`. The repository branch listing places it first among the branches shown as updated four hours earlier, and its source tree contains the DCF engine, scenario CRUD, database integration, comparison UI, and authentication foundation. In contrast, the adjacent `bhavin/auth` and `bhoomi/scenarios` branches still expose the earlier Week-1 frontend scaffold. The implementation below therefore reflects `demo/comparison` rather than `master`.

2.1.1 Use Case Diagram

The use case diagram in Figure 2.1 identifies the actors of the system and the functionality available to each. A *User* can enter assumptions, fetch company data by ticker, compute intrinsic value, and analyze sensitivity. A *Registered User* specializes the User and can additionally save, load, delete, and compare scenarios, which requires logging in. Fetching company data involves the external *Financial Data API*, always includes validation of the returned data, and is extended by a fallback to manual entry when that data is unusable.

2.1.2 Class Diagram

Figure 2.2 presents the domain model defined in `frontend/src/domain/scenario.ts` and mirrored field-for-field in `backend/app/models.py`. A `Scenario` is composed of exactly one `Assumptions` object and optionally aggregates one `ValuationOutput`, which is in turn composed of zero or more `ProjectedYear` records. The `runDCF()` dependency links the assumptions to the valuation output they produce. The enumerations `ScenarioKind`, `DataStatus`, and `ValuationStatus` constrain the scenario type, data provenance, and computation result respectively.

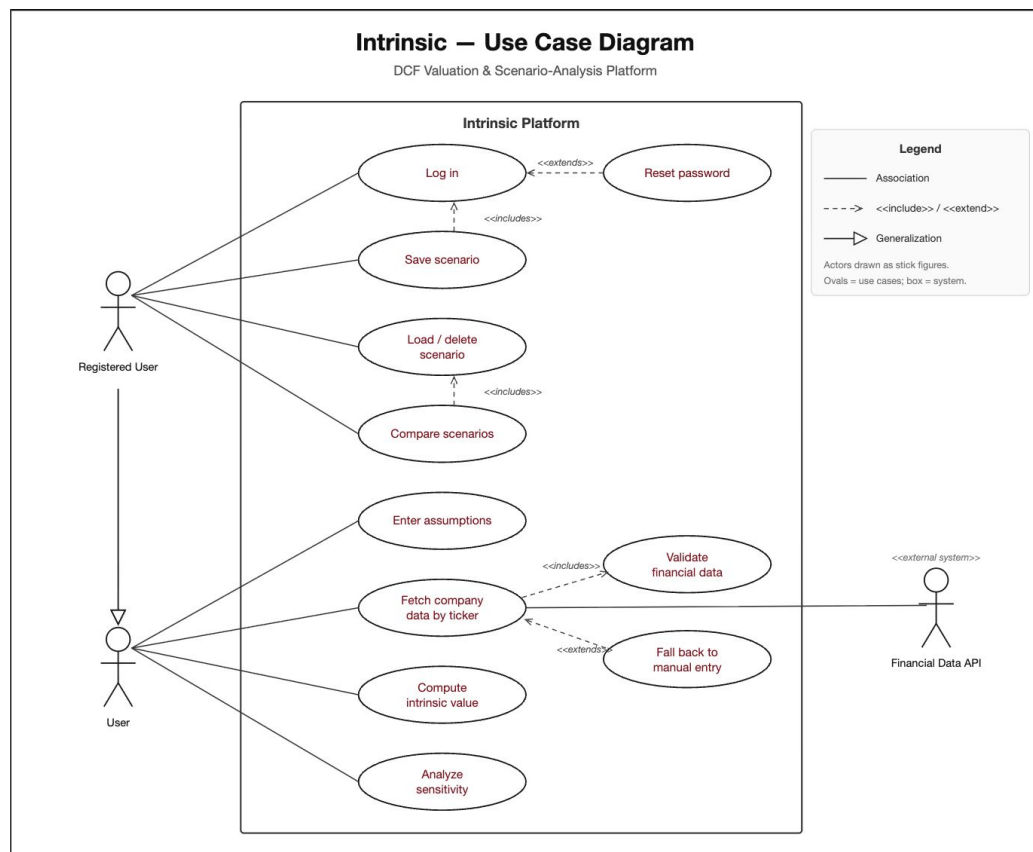
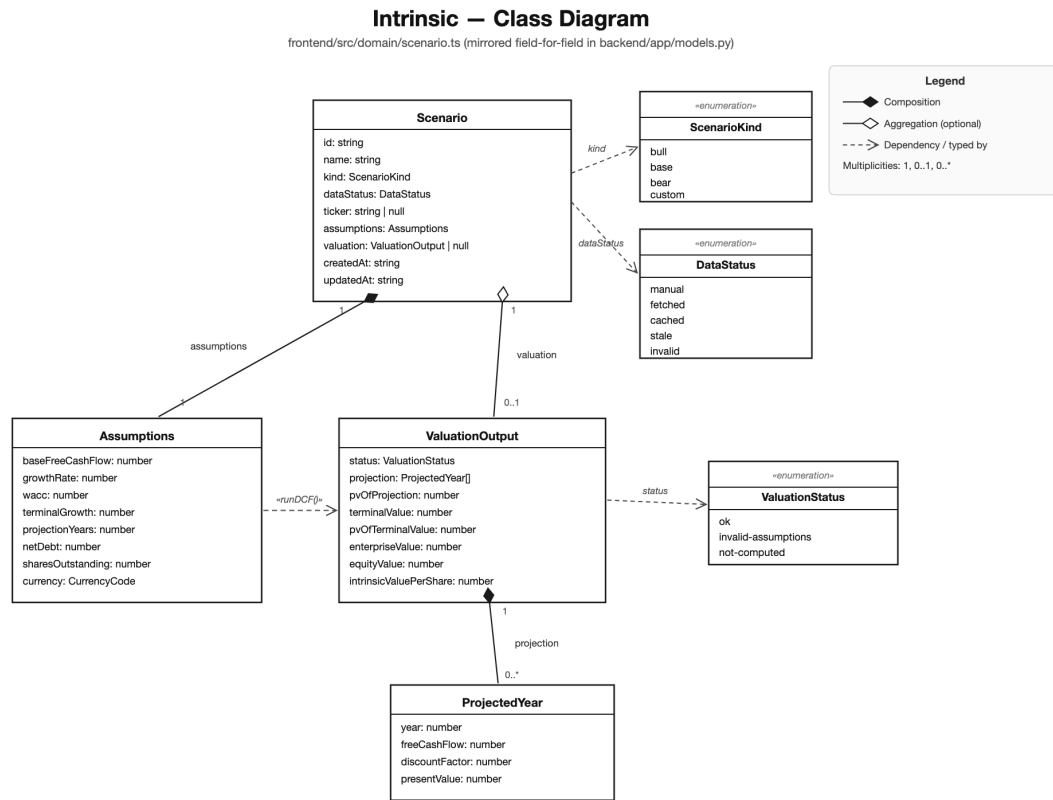


Figure 2.1: Use case diagram of the Intrinsic DCF valuation system.



Pydantic mirror (backend/app/models.py) uses snake_case field names for the same shapes; TypeScript is the source of truth.

Figure 2.2: Class diagram of the Intrinsic domain model.

2.1.3 Sequence Diagram

Figure 2.3 shows the interaction between the User, Frontend, DCF Engine, Backend, and external Financial Data API across a complete valuation. When a ticker is supplied, the backend fetches and validates the raw data, returning either the assumptions (`dataStatus = fetched`) or a validation error that causes the frontend to fall back to manual entry. After the user edits the assumptions, the frontend invokes `runDCF`; the guardrail fragment shows the two possible outcomes depending on whether terminal growth is below WACC. Finally, the scenario is posted to the backend, persisted with an identifier and timestamps, and confirmed to the user.

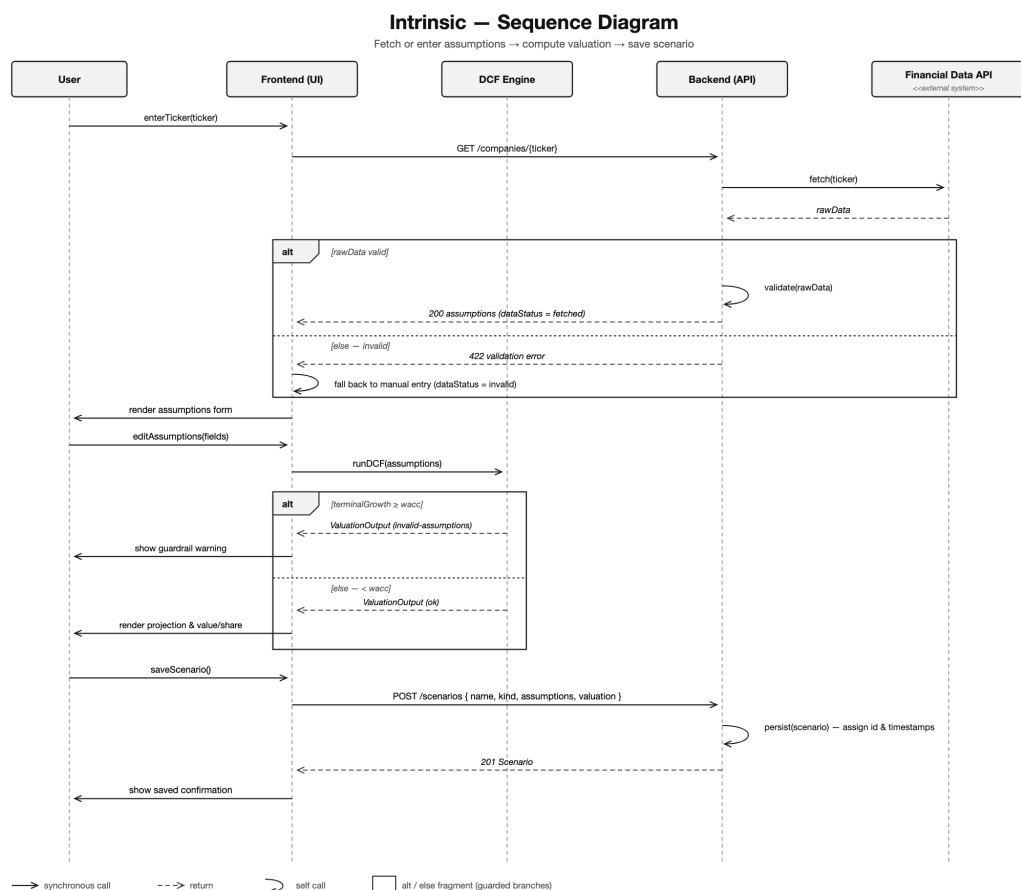


Figure 2.3: Sequence diagram for fetching or entering assumptions, computing the valuation, and saving a scenario.

2.1.4 Data Flow Diagrams

Figure 2.4 presents the data flow of the system at two levels. The Level 0 context diagram treats Intrinsic as a single process exchanging data with the User and the Financial Data API. The Level 1 diagram decomposes it into four processes: (1.0) Manage Assumptions, (2.0) Fetch & Validate Financial Data, (3.0) Compute Valuation, and (4.0) Manage Scenarios. Two data stores support these processes: D1 Scenario Store backs save, load, and compare operations, while D2 Data Cache holds normalized external data as part of the planned Month-2 data-integration work.

2.1.5 Swimlane Diagram

The swimlane diagram in Figure 2.5 assigns each step of the *Perform Valuation* workflow to the component responsible for it: User, Frontend, DCF Engine, Backend, and Financial Data API. It makes the hand-offs between components explicit, including the ticker/manual-entry decision, the backend data-validity check with fallback, the terminal-growth guardrail with its retry loop, and the optional save path that ends in a persisted scenario and a confirmation.

2.1.6 Activity Diagram

Figure 2.6 details the internal control flow of the `runDCF` function in `frontend/src/domain/runDCF`. The function first checks the guardrail $g_T \geq WACC$ and returns a zeroed output with status `invalid-assumptions` if it is violated. Otherwise, it loops over each projection year, computing the free cash flow, discount factor, and present value and appending them to the projection. After the loop, it computes the present value of the projection, the terminal value and its present value, enterprise value, equity value, and intrinsic value per share, and returns the output with status `ok`.

2.1.7 High-Level Design

The main runtime flow is shown in Figure 2.7. A user edits the valuation assumptions in the React interface. The frontend invokes the pure `runDCF` function locally, producing a structured `ValuationOutput`. Saved scenarios are sent to the FastAPI scenario API as JSON. The backend stores the assumptions and valuation output in a `SQLModel/SQLite` database and returns them in a frontend-compatible shape. The comparison component then consumes multiple saved scenarios to produce a side-by-side metric table, relative bars for intrinsic value per share, and a valuation range.

The architecture deliberately separates calculation from persistence. This means the DCF engine does not require a database connection, HTTP request, React state, or browser APIs. Conversely, the scenario API does not reimplement the financial

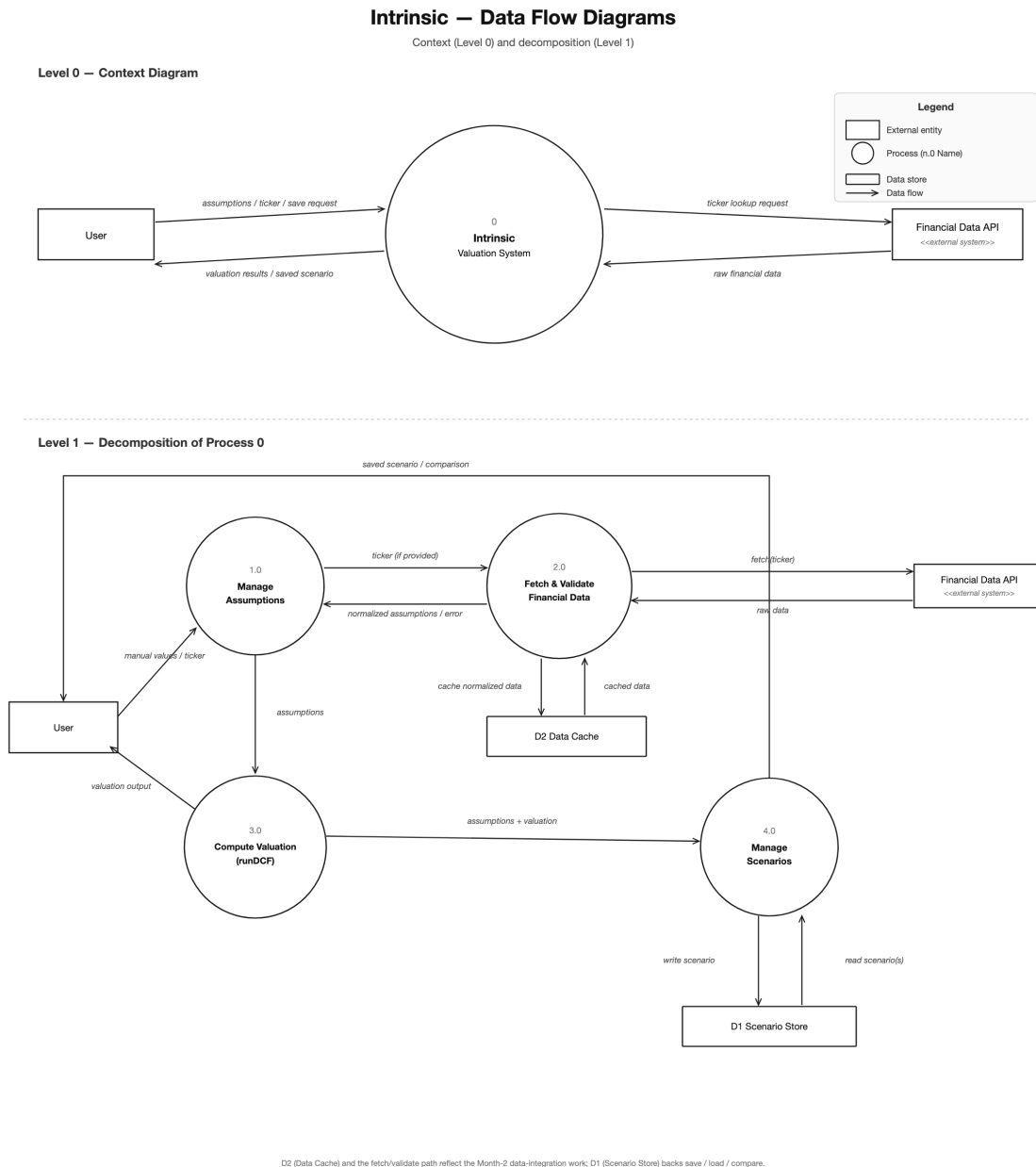


Figure 2.4: Data flow diagrams of Intrinsic: Level 0 (context) and Level 1 (decomposition).

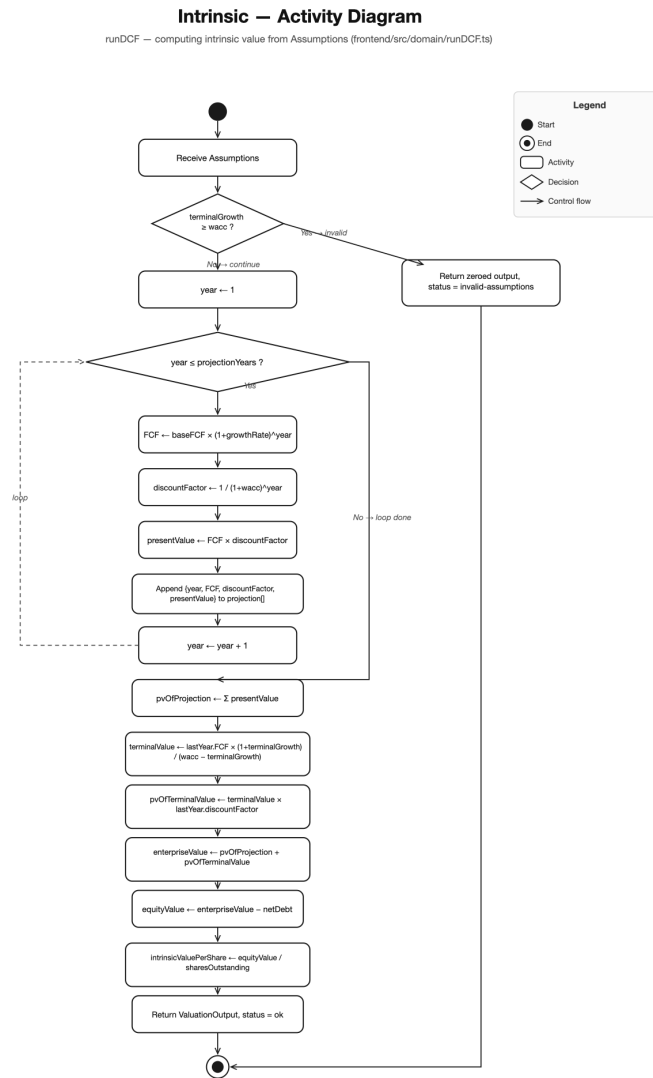


Figure 2.6: Activity diagram of the runDCF valuation computation.

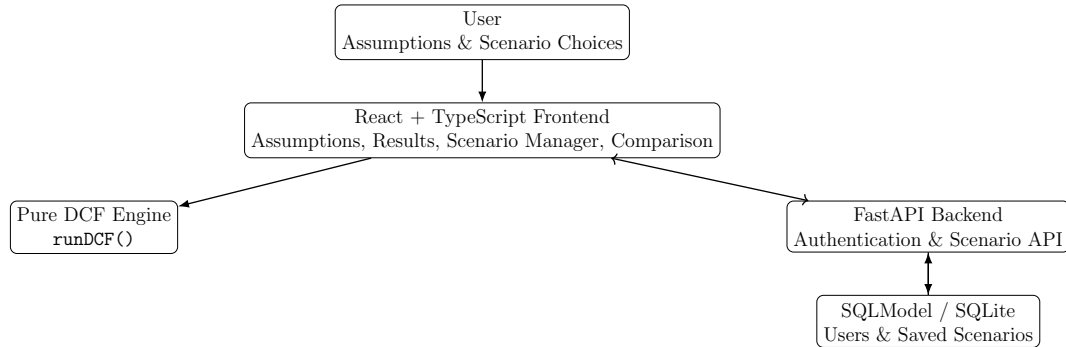


Figure 2.7: High-level architecture of the current `demo/comparison` branch.

mathematics; it stores and retrieves the already-structured valuation representation. This separation gives the project a clear boundary between domain logic and application infrastructure.

2.1.8 Technical Stack

- **Frontend:** React with TypeScript, Vite, Tailwind CSS, and Vitest. The frontend is organized into API helpers, domain models, valuation logic, and presentation components.
- **Backend:** FastAPI on Python, with Pydantic models for API contracts and SQLModel for persistence.
- **Database:** SQLite in the current implementation. Scenario assumptions and valuation outputs are stored as JSON fields so that the frontend scenario shape can round-trip without lossy transformation.
- **Authentication:** bcrypt-based password hashing and JWT access tokens, exposed through registration, login, and current-user endpoints.
- **Testing and quality:** Vitest tests the DCF engine, while the backend uses pytest and Ruff. The project also contains CI support for automated checks.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core functionality is the complete local valuation workflow: collect a typed set of assumptions, calculate the DCF deterministically, render the intermediate and final outputs, and preserve the resulting analysis as a named scenario that can later be compared.

1. Scenario and assumption model. The frontend defines a shared scenario contract in `frontend/src/domain/scenario.ts`. The `Assumptions` structure contains the latest free cash flow, annual growth rate, WACC, terminal growth rate, explicit projection period, net debt, diluted shares outstanding, and currency. Rates are represented as decimals, while monetary values are expressed in millions of the selected currency. A `Scenario` additionally contains a stable identifier, human-readable name, scenario kind (Bull/Base/Bear/Custom), data-status metadata, optional ticker, timestamps, and an optional computed valuation.

This model is the conceptual centre of the system. It allows the same valuation state to travel through the UI, API, and persistence layers without changing meaning. The backend Pydantic models mirror the same logical fields and are explicitly documented as a field-for-field contract.

2. DCF computation. The valuation engine is implemented in `frontend/src/domain/runDCF.ts`. It is pure and side-effect-free: the same assumption set always produces the same result and the input object is not mutated. For each projected year t , the engine calculates

$$FCF_t = FCF_0(1 + g)^t,$$

$$DF_t = \frac{1}{(1 + WACC)^t},$$

and

$$PV(FCF_t) = FCF_t \times DF_t.$$

The explicit-period present value is the sum of the yearly present values. Terminal value is calculated using the Gordon Growth formulation,

$$TV = \frac{FCF_n(1 + g_T)}{WACC - g_T},$$

and discounted back using the final year's discount factor. Enterprise value is then

$$EV = PV_{projection} + PV_{terminal},$$

followed by

$$Equity\ Value = EV - Net\ Debt,$$

and finally

$$Intrinsic\ Value\ per\ Share = \frac{Equity\ Value}{Shares\ Outstanding}.$$

The engine enforces the domain guardrail $g_T < WACC$. When this is violated, the engine returns an `invalid-assumptions` status instead of producing a meaningless terminal value. This is important because the terminal-value denominator would otherwise become zero or negative.

3. Results presentation. The `ResultsView` component renders both the projection table and the valuation roll-up. The projection table exposes year, free cash flow, discount factor, and present value. The roll-up then shows the present value of the projection period, terminal value, present value of terminal value, enterprise value, equity value, and intrinsic value per share. Thus, the interface makes each important intermediate stage visible rather than hiding all calculation steps behind a final figure.

4. Scenario persistence. The `ScenarioManager` provides the persistence workflow. A user assigns a scenario name and a qualitative case type, then saves the current assumptions and valuation through `POST /api/scenarios`. Existing scenarios can be listed, loaded back into the assumption workspace, or deleted. The frontend API wrapper is intentionally small: it serializes the typed scenario payload, calls the REST endpoint, and parses the JSON response back into the same domain shape.

On the backend, the scenario router exposes three corresponding operations: list, create, and delete. Scenario records contain JSON columns for the assumptions and valuation, together with metadata such as name, type, ticker, and timestamps. The current demo leaves scenario endpoints open rather than attaching scenarios to authenticated users, which keeps the comparison demonstration simple while the authentication foundation is developed separately.

5. Scenario comparison. The `ComparisonView` consumes the saved scenarios and compares them across growth rate, WACC, terminal growth, enterprise value, and intrinsic value per share. It also renders a relative bar representation for intrinsic value per share and computes the observed valuation range and spread. The component requires at least two scenarios before a comparison is displayed, matching the intended Bull/Base/Bear workflow.

6. Authentication foundation. The backend additionally contains an authentication slice with registration, login, and current-user endpoints. Passwords are hashed with `bcrypt` rather than stored directly, and login produces a JWT access token containing the user identifier and an expiry time. This is a supporting module rather than a prerequisite for the current demo scenario comparison, because the scenario router explicitly remains open in the present implementation. Its architectural purpose is to establish the boundary needed for future per-user scenario ownership.

7. Persistence architecture. The database layer is implemented with `SQLModel` and `SQLite`. The database engine is created centrally, sessions are injected through `FastAPI` dependencies, and application startup creates the registered `SQLModel` tables. The persistent schema contains a `User` table and a `Scenario` table. The scenario table stores the structured assumptions and valuation as JSON so the application can

preserve the frontend representation without manually decomposing every nested field into separate columns.

8. Testing of the core module. The DCF engine has a hand-worked regression test that checks the calculation year by year and through the final roll-up. The test verifies a three-year example whose expected intrinsic value per share is 152.5, checks determinism and non-mutation, and checks that terminal-growth values equal to or greater than WACC are rejected. This is a strong match to the project's stated engineering goal: the financial formula is standard, while correctness is established through explicit, repeatable tests.